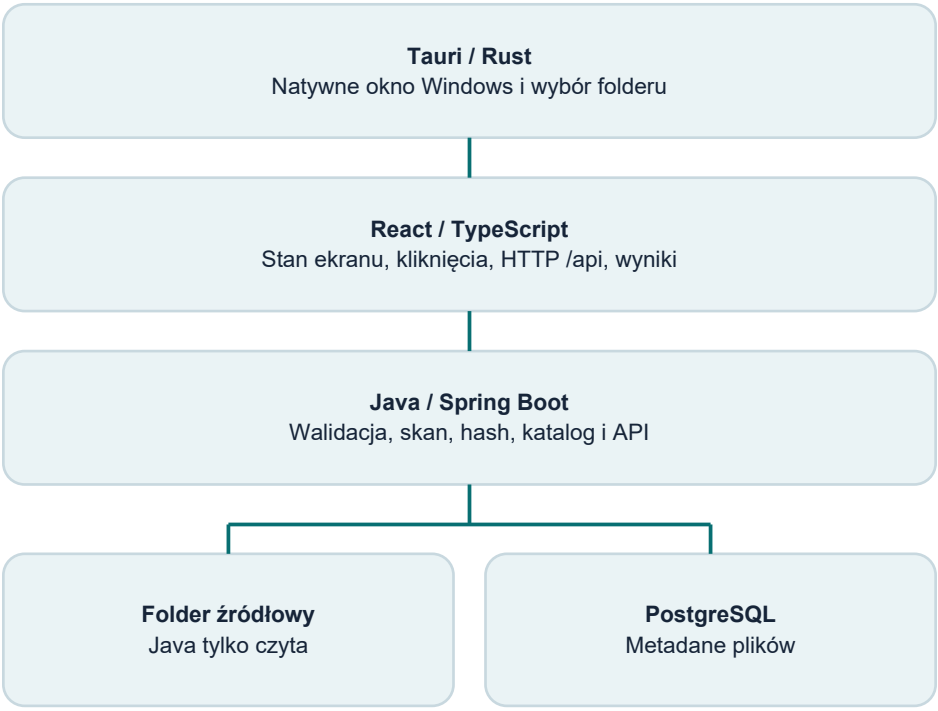
Mapa architektury ScreenSort

Stan obecny. Diagram w pionie, od aplikacji desktopowej do źródłowych plików i bazy.



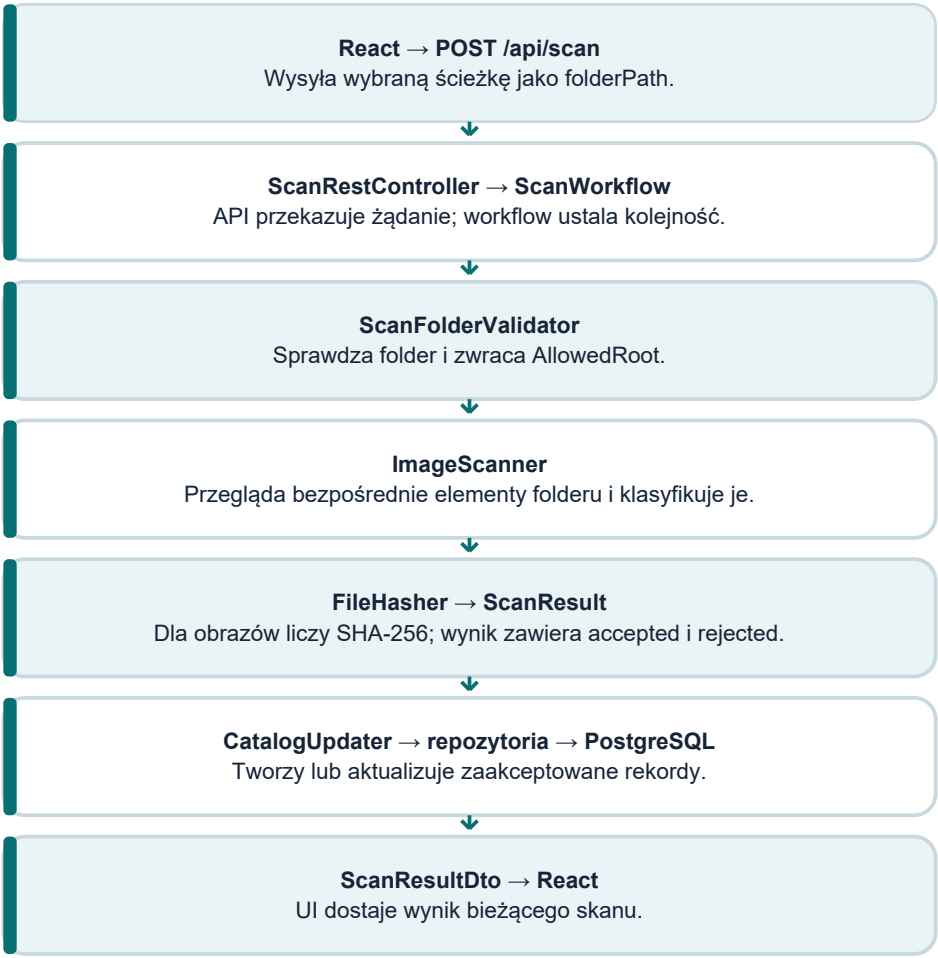
Granice odpowiedzialności

Tauri otwiera okno i natywny picker. **React** przechowuje stan widoku i wysyła żądania. **Java** czyta pliki, decyduje co przyjąć i zapisuje metadane. PostgreSQL nie zawiera kopii zdjęć.

W obecnym trybie developerskim Tauri nie startuje automatycznie Javy ani bazy. Wybranie folderu samo w sobie nie uruchamia skanu.

Skanowanie i zapis

Ścieżka jednego kliknięcia Scan folder przez istniejące klasy Javy.



Odrzucone pliki trafiają do odpowiedzi, ale nie są zapisywane jako file_record. Oryginalne obrazy pozostają bez zmian.

Odczyt katalogu

To osobne żądanie; nie wykonuje ponownego skanu zdjęć.

React → GET /api/catalog/files
Pyta o zapisane pliki wybranego folderu.

↓

CatalogRestController → CatalogReader
Przekazuje ścieżkę do odczytu.

↓

Walidator + repozytoria JPA
Sprawdza folder i pobiera jego rekordy z PostgreSQL.

↓

CatalogFileDto[] → React
Saved images pokazuje trwałe metadane.

Nie pomyl tych obiektów

Obiekt	Rola
AllowedRoot	Zweryfikowany folder w pamięci Javy.
AcceptedImage	Dane jednego obrazu znalezione podczas skanu.
ScanResult	Wynik jednego bieżącego skanowania.
FileRecordEntity	Reprezentacja trwałego wiersza bazy.
CatalogFileDto	Dane z katalogu wysłane do UI.

Kod i diagramy Mermaid: docs/diagramy-architektury.md. Dwa wcześniejsze PNG są w app-workflow-images/.